



DATA STRUCTURES

CONTENTS



28. Multi-Way Trees and 2-3 Trees

TV. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. We have understood the basic concept of balanced trees and discussed AVL trees and Red Black trees, both balanced binary search trees. We also discussed a self-balancing binary search tree, the Splay tree. In this module, we will discuss trees that are not necessarily binary that is multi-way search trees in general and 2-3 trees in particular.

Learning Objectives

The learning objectives of the module are as follows:

- To understand the idea of Multi-way SearchTrees
- To discuss the concept of 2-3 Trees
- To describe the operations on 2-3 Trees

28.1 Multi-Way Search Trees

Multi-Way search trees are not binary search trees because they are not necessarily binary. These trees maintain all leaves at same depth, but number of children can vary. In general an m-way search tree is a tree in which, for some integer m called the order of the tree, each node has at most m children. For any node in the tree, if $k \leq m$ is the number of children, then the node contains exactly $k - 1$ keys, which partition all the keys into k subsets consisting of all the keys less than the first key in the node, all the keys between a pair of keys in the node, and all keys greater than the largest key in the node. Every non-leaf node has either 2 or 3 children. All leaves are at the same depth. Information (keys and associated data) is stored **only** at leaves. The internal nodes are for organization only. The keys at leaves are ordered left to right. In addition to child pointers, each internal node stores the value of the max key in the **left** subtree (leftMax) and the value of the max key in the **middle** subtree (middleMax) if the node has 3 children. If a node only has 2 children, they are left and middle (not left and right) children.

When a tree has 2 or 3 children they are called 2-3 tree and when they have 2, 3, or 4 children they are called 2-3-4 tree while we also have the B-tree where the number of children are approximately between $B/2$ to B children. All multi-way search trees are balanced search trees. We will discuss this aspect later. Let us discuss in detail the 2-3 Trees where each internal node has either 2 or 3 children. In this tree all leaves are at the same level. These trees can be perfectly balanced where there is no difference in height between branches. Figure 28.1 shows an example of a 2-3 Tree.

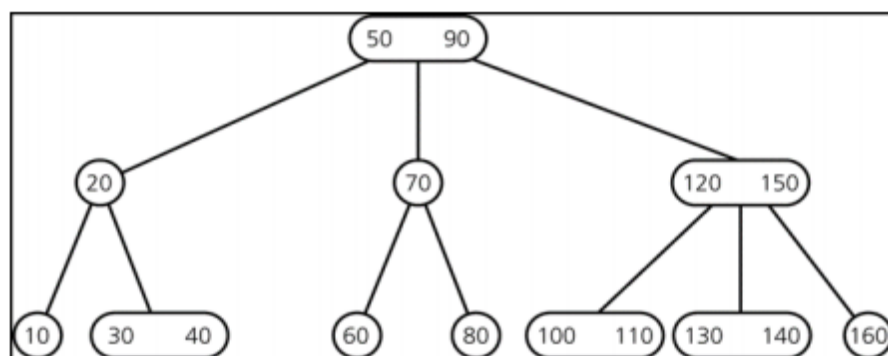


Figure 28.1 Example of a 2-3 Tree

2-3 Trees are *multiway trees* here each node has at most 3 children, and it can hold up to 2 values. However these trees require complex nodes and links . In the example the root has 2 keys 50 and 90 and 3 sub-trees. All keys less than 50 are in the left sub-tree, between values 50 and 90 and in the middle sub-tree and all keys greater than 90 are in the right sub-tree. This true for all internal nodes. All the information is at the leaves.

2-3-4 Trees are also *multiway trees* where each node has at most 4 children, and a node can hold up to 3 values. Figure 28.2 shows an example of a 2-3-4 tree. In the example the root has 3 keys 60, 75 and 90 and 4 sub-trees. All keys less than 60 are in the first sub-tree, between values 60 and 75 in the second sub-tree, between 75 and 90 in the third sub-tree and all keys greater than 90 are in the fourth sub-tree.

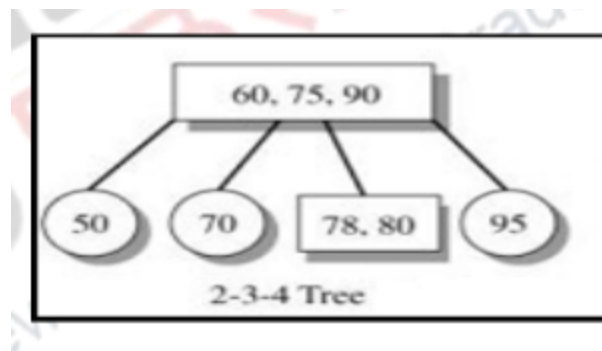


Figure 28.2 Example of a 2-3-4 Tree

B-Trees are also multiway tree where each node has at most m children, and a node can hold up to $m-1$ values. This tree is a more general version of 2-3 and 2-3-4 trees. B-Trees are commonly used in databases and file systems where most nodes are stored in secondary storage such as hard drives . Figure 28.3 shows an example of a B tree with at most 5 children.

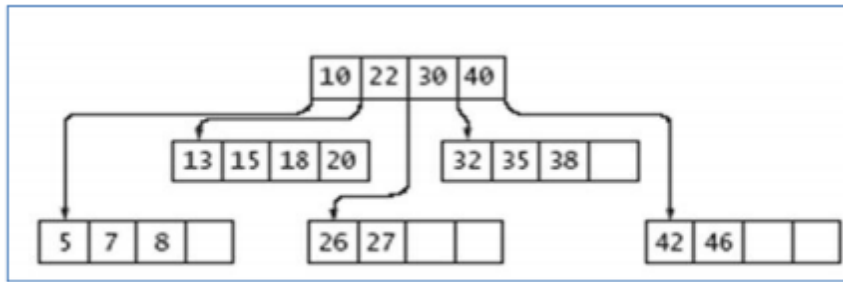


Figure 28.3 Example of a B-Tree

28.2 2-3 Trees

In 2-3 trees each internal node has either 2 or 3 children and all leaves are at the same lowest level. 2-3 tree has been named based on the number of possible children of each node. Depending on the number of children each node is designated as either a 2-node or a 3-node. A 2-node is the same as a binary search tree node with values attached to it as shown in Figure 28.1. Figure 28.3 shows a basic structure of a 2-3 Tree.

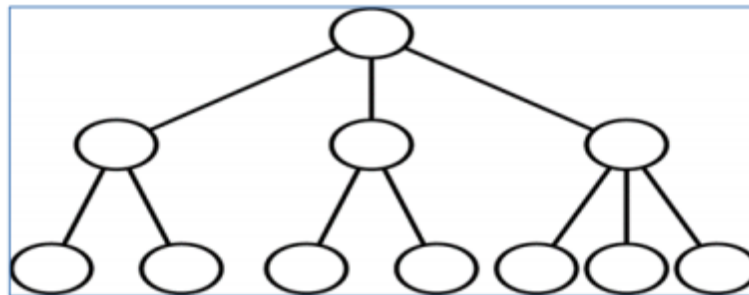


Figure 28.4 Structure of a 2-3 Tree

Now let us discuss the properties of a 3-node. The 3-node has three sub-trees associated with it with and references to three children. The 3-node contains *two keys or data fields*, where the first data field is lesser than the second data filed. The first sub-tree holds data values which are less than the first field, the second sub-tree holds values between the first and second data fields, while the third sub -tree holds values greater than the second data field.

All non-leaf nodes have either 1 key and two sub trees, or 2 keys and three sub trees. Insertion is always at the leaf. If the leaf (which can hold a maximum of 2 values) overflows, we split it into two leaves, insert them into the parent, which in turn may also overflow. If the deletion leads to a leaf underflow (has no items), we merge it with a sibling, removing a value and sub tree from the parent, which may in turn also underflow. The only changes in depth of the tree happens when the root splits or underflows

28.2.1 2-3 Tree: Definition

Formally, T is a 2-3 tree of height h if

- a) T is **empty** ($h = 0$); or
- b) T consists of a root and **2 subtrees** (Every non-leaf node has either 2 or 3 children), T_L , T_R :

- T_L and T_R are both 2-3 trees, each of height $h - 1$,
- the root contains **one** data item with search key S ,
- $S >$ each search key in T_L ,
- $S <$ each search key in T_R ;

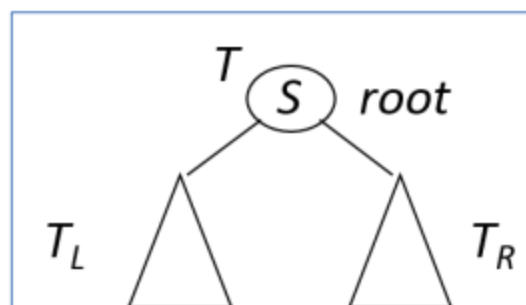


Figure 28.5 2-node

- c) T consists of a root and **3 subtrees**, T_L , T_M , T_R :

- T_L , T_M , T_R are all 2-3 trees, each of height $h - 1$,
- the root contains **two** data items with search keys S and L ,
- each search key in $T_L < S <$ each search key in T_M ,
- each search key in $T_M < L <$ each search key in T_R .

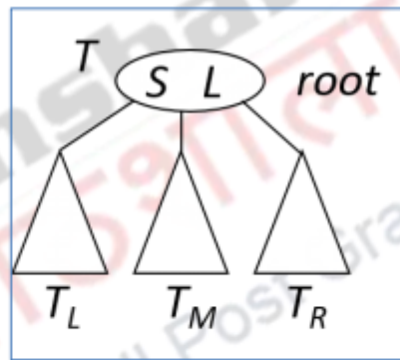


Figure 28.6 3-node

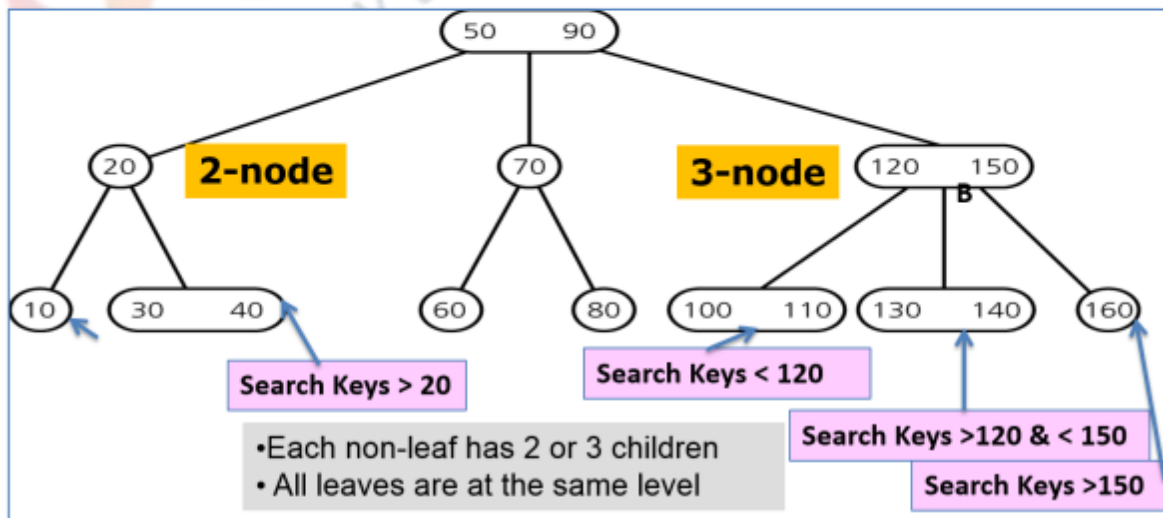


Figure 28.7 Another Example of a 2-3 Tree

Let us look at the example in Figure 28.7. The root is a 3-node that has two data values 50 & 90 and three sub-trees. The leftmost sub-tree having a 2-node root (with one data value 20) has all data values < 50 . The middle sub-tree having again a 2-node root has all data > 50 but < 90 . The rightmost sub-tree having a 3-node root has all data > 90 . As can be seen each non leaf

28.3 2-3 Tree: Efficiency

Insertion and deletion can be defined so that the 2-3 tree remains balanced and its other properties are maintained. In particular, a 2-3 tree with n values *never* has a height greater than $\lceil \log_2 (n + 1) \rceil$, same as the *minimum* height of a binary tree with n nodes or values. Searching an item in a 2-3 tree is *never worse than* $O(\log n)$,

regardless of the insertions or deletions that were done previously. Traversing a 2-3 tree is similar to in order traversal of a search tree.

28.4 2-3 Tree Insertion

28.4.1 The Algorithm

Here we refer to the values at the internal nodes as leftMax and middleMax in case of 2-node and leftMax, middleMax and rightMax in case of 3- node.

1. Handling of empty trees and tree with single node

if T is empty replace it with a single node containing k else if T is just 1 node m:
create a new leaf node n containing k
create a new internal node with m and n as its children,
and with the appropriate values for leftMax and middleMax
else call ninsert(T, k)

2. ninsert method to handle other cases

The ninsert method is the recursive method that handles all but the 2 special cases .

2a. Finding parent of the new node to be inserted

As is the case for binary-search trees, the first task of the ninsert method is to find the (non-leaf) node that will be the **parent** of the newly inserted node. The ninsert method performs the following steps to find node n, the parent of the new node:

- base case: T's children are leaves – n is found! (T will be the parent of the new node)
- recursive cases:
- $k < T.\text{leftMax}$: insert k into T's left subtree
- $T.\text{leftMax} < k < T.\text{middleMax}$, or T only has 2 children: insert k into T's middle subtree

- $k > T.\text{middleMax}$ and T has 3 children: insert k into T 's right subtree. Once n is found, there are two cases, depending on whether n has room for a new child:

2b. Case 1: n has only 2 children

- Insert k as the appropriate child of n :
 1. if $k < n.\text{leftMax}$, then make k n 's left child (move the others over), and fix the values of $n.\text{leftMax}$ and $n.\text{middleMax}$. Note that all ancestors of n still have correct values for their leftMax and middleMax fields (because the new value is not the "max" child of n).
 2. if k is between $n.\text{leftMax}$ and $n.\text{middleMax}$, then make k n 's middle child and fix the value of $n.\text{middleMax}$. Again, no ancestors of n need to have their fields changed.
 3. if $k > n.\text{middleMax}$, then make k n 's right child and fix the leftMax or middleMax fields of n 's ancestors as needed.

2c Case 2: n already has 3 children

- Make k the appropriate new child of n , anyway (fixing the values of $n.\text{leftMax}$ and/or $n.\text{middleMax}$ as needed). Now n has 4 children.
- Create a new internal node m . Give m n 's two rightmost children and set the values of $m.\text{leftMax}$ and $m.\text{middleMax}$.
- Add m as the appropriate new child of n 's parent (i.e., add m just to the right of n). If n 's parent had only 2 children, then stop creating new nodes, just fix the values of the leftMax and middleMax fields of ancestors as needed. Otherwise, keep creating new nodes recursively up the tree. If the root is given 4 children, then create a new node m as above, and create a new root node with n and m as its children.

28.4.2 Tree Insertion Example

Let us consider the Example 1 given in Figure 28.8. Here we need to insert 37 into the 2-3 tree. We see that insertion of 37 is a case of step 2 of the steps given in section 28.4.1. Here insertion results in the leaf having only 2 data items, so the

process is over. Figure 28.9 shows Example 2 where we insert 36 into the tree obtained after insertion in Example 1. Now insertion of 36, makes the middle child of the root to have

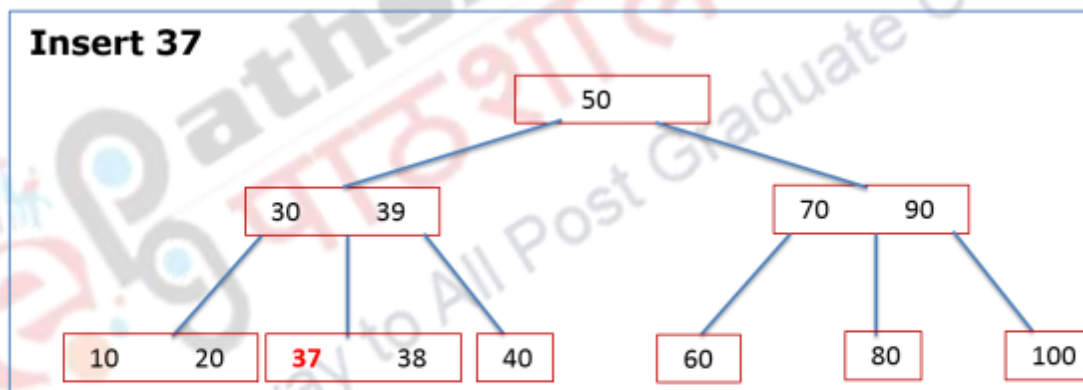
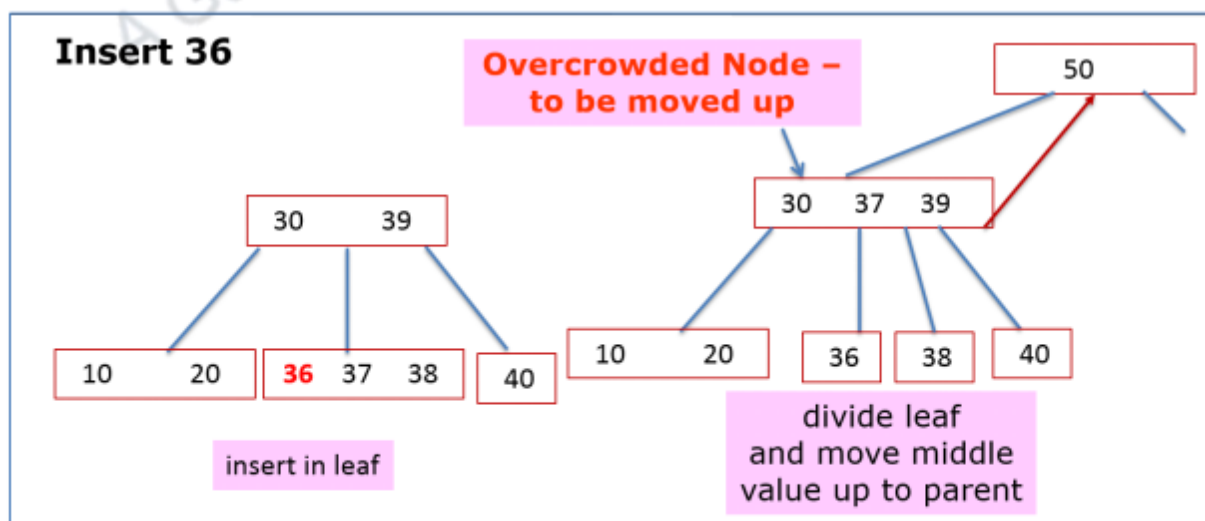
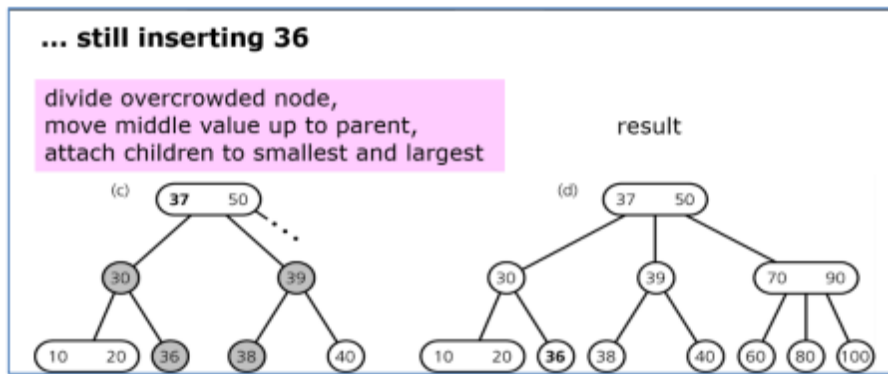


Figure 28.8 Insertion into 2-3 Tree – Example 1



(a)



(b)
Figure 28.9 Insertion into 2-3 Tree – Example 2

three data values and this is not allowed in 2-3 tree as shown in (Figure 28.9 (a)). The leaf now contains 3 items, 36, 37, & 38. Therefore we need to divide leaf and move middle value up to parent. We *replace* the leaf by two new nodes, n_1 and n_2 , with the *smallest* of 36, 37, 38 i.e. 36 being n_1 , the *largest* i.e. 38 being n_2 , and the *middle* value (that is 37) going into the leaf's parent node, (30, 39). We now need to make the nodes 36 and 38 the children of the parent (30, 37, 39). This insertion of the middle value makes the parent to have three data values and hence overcrowded. Now parent p contains 3 items (30, 37 & 39) as shown in (Figure 28.9 (b)) and 4 children ((10, 20), 36, 38 and 40). We proceed except that p 's two leftmost children ((10 and 20) and 36) are attached to 30, and p 's two rightmost children (38 and 40) are attached to 39. Figure 28.10 shows the final 2-3 Tree after insertion of 35, 34 and 33.

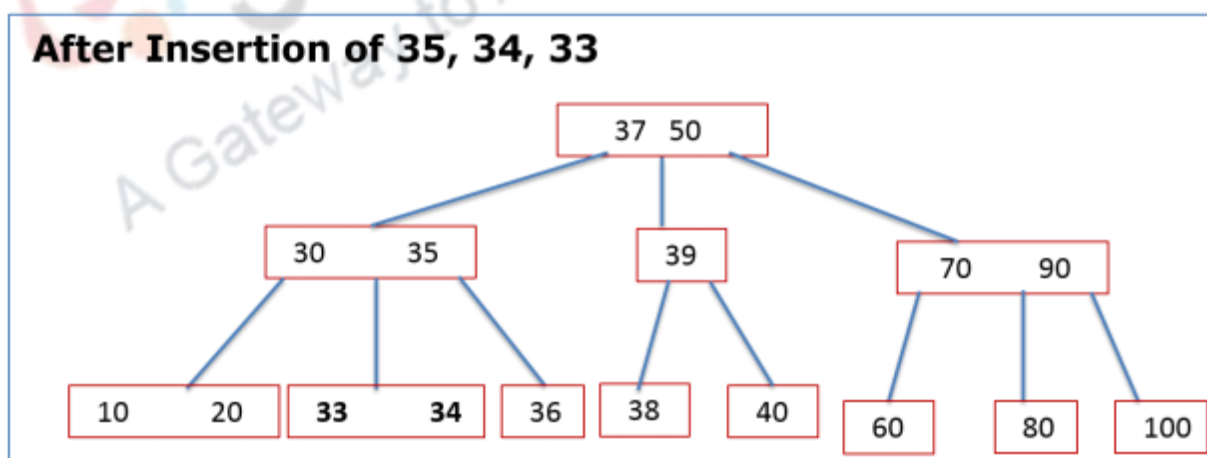


Figure 28.10 Final tree after insertion of 35, 34 and 33

28.4.3 Explaining the Insertion

In order to insert an item I into a 2-3 tree, we first locate the leaf at which the search for I would terminate. Then we insert the new item I into the leaf. If the leaf now contains only two items, you are done. If the leaf contains three items, you must split it. When an internal node contains three items we split the node into two nodes in order to accommodate the node's children. When the root contains three items, we split the root into two nodes and create a new root node (Figure 28.11).

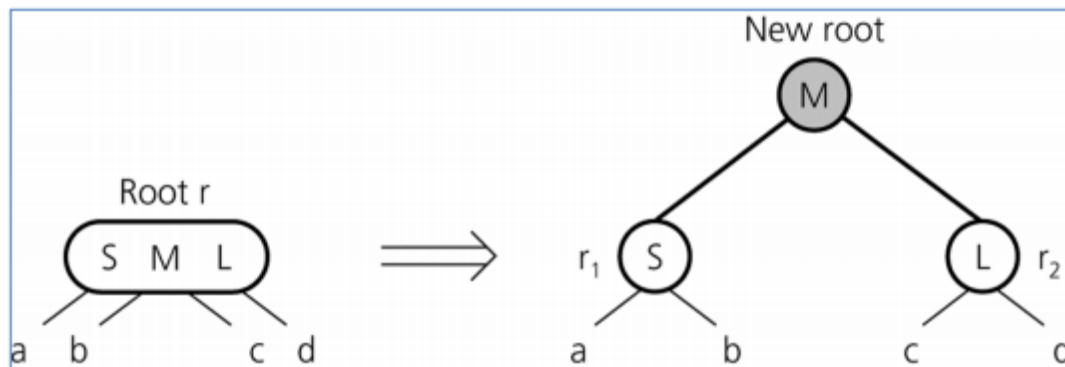


Figure 28.11 Insertion when Root contains Three Nodes

The splitting of a leaf is shown in Figure 28.12. Here we assume P has only two children. If the left leaf of parent P gets three data values S,M and L then after splitting, M moves up to become the data value added to the left of P. Then leaf with S and L are split to become 2 new children of the parent node (M,P). On the other hand if the right leaf of parent P gets three data values S,M and L then after splitting, M moves up to become the data value added to the right of P. S and L are split to become 2 new children of the parent node (P,M) .

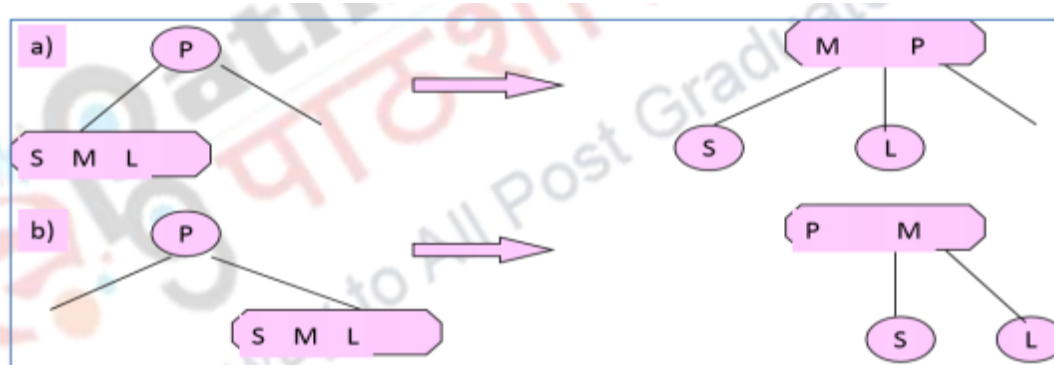
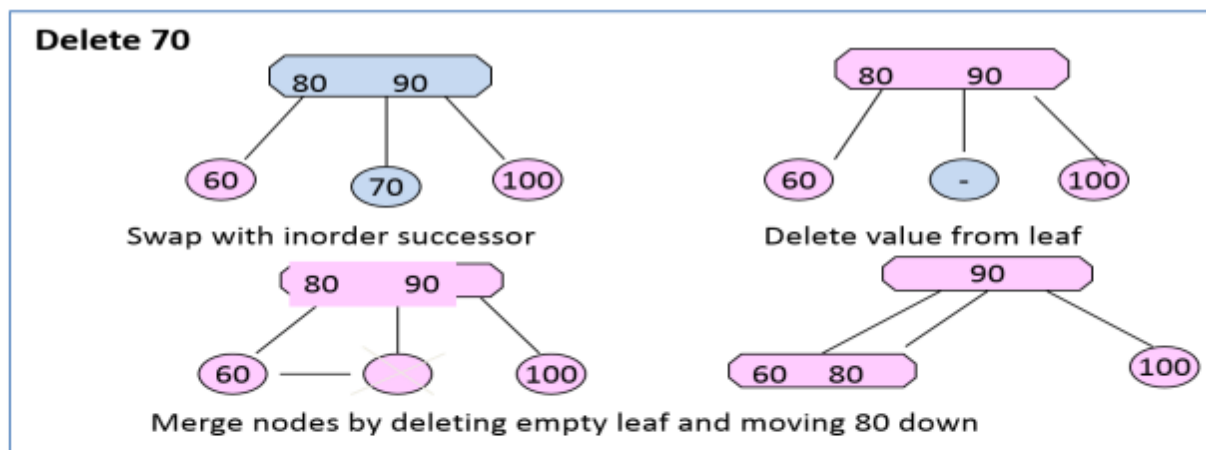


Figure 28.12 Splitting in Insertion

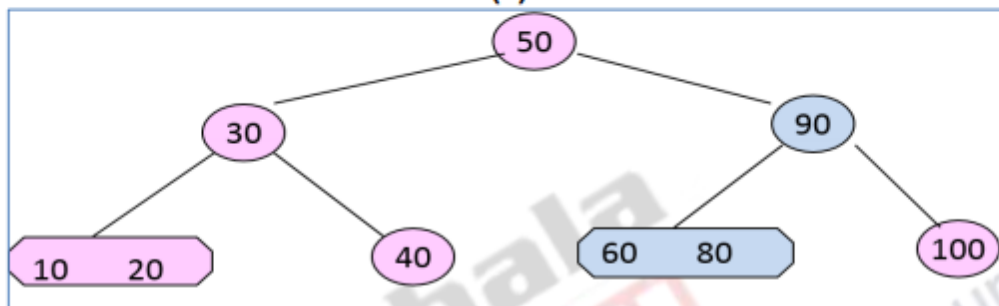
The goal of the insert operation is to insert key k into tree T , maintaining T 's 2-3 tree properties. Special cases are required for empty trees and for trees with just a single (leaf) node. Therefore let us start with these special cases:

28.5 Deletion Algorithm

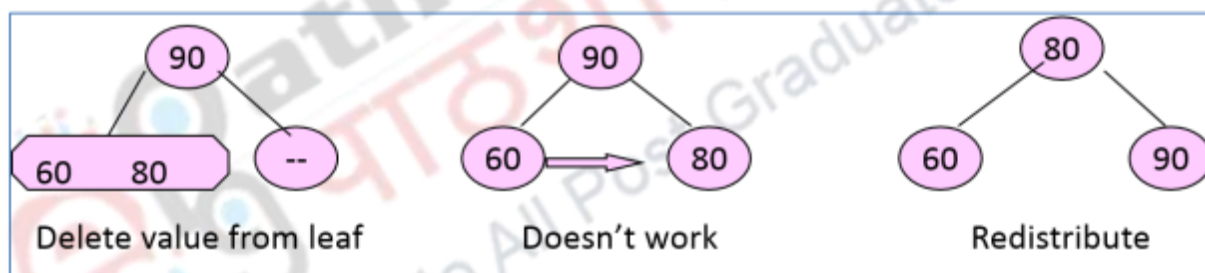
The deletion strategy for a 2-3 tree is the inverse of its insertion strategy. While the 2-3 tree spreads insertions throughout the tree by splitting nodes when they become too full, deletion spreads deletions throughout the tree by merging nodes when they become empty.



(a)



(b)



(c)

Figure 28.13 Deletion from 2-3 Tree

Figure 28.13 shows the process of deletion. Figure 28.13 (a) shows the deletion of 70. This leaves an empty leaf, so the parent has now two children only so one of the data values (80) needs to move down. Now 80 is merged with 60 to get a new child (60,80). Therefore the children of parent (90) now becomes (60,80) and (100). Now

let us assume we delete the leaf 100. Now we need to split the (60,80) node to become two children (60) and (80) of parent 90. But this does not work and we need rearrange the values as shown in the figure.

28.5.1 Deleting a Leaf

When we delete a leaf with a single data value, its left sibling if it contains 2 data values (S,L) is split and redistributed (Figure 28.14). Now if the leaf to be deleted has a left sibling with single child value (S) , then the parent P is merged with S.

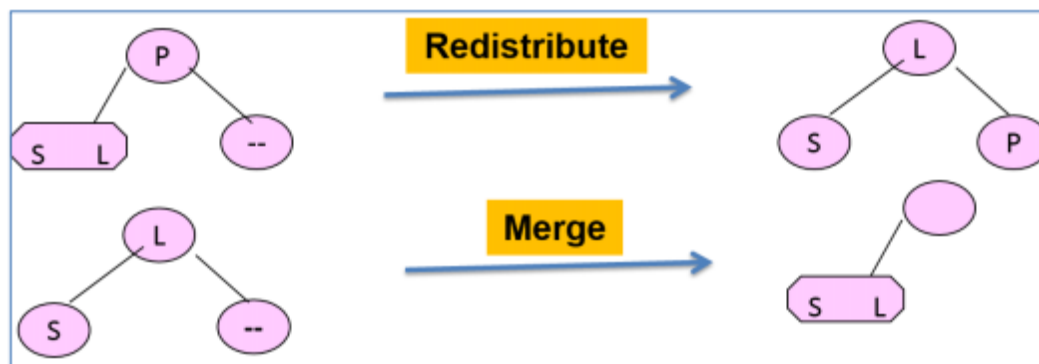


Figure 28.14 Deleting a Leaf

28.5.2 Deleting Internal Nodes

When we delete an internal node with only child **d** (Figure 28.15) we can redistribute if the left sibling of the parent of the node to be deleted has three children. The three children (**a,b,c**) of its left sibling are redistributed such that **L** is moved up the tree and **S** and **P** now are children of **L**. Both **S** and **P** have two children each (**a,b**) and (**c,d**) respectively. However if the left sibling of the parent of the node to be deleted has only two children, we cannot redistribute the children. In this case, the node is deleted and its left sibling and parent are merged (Figure 28.15).

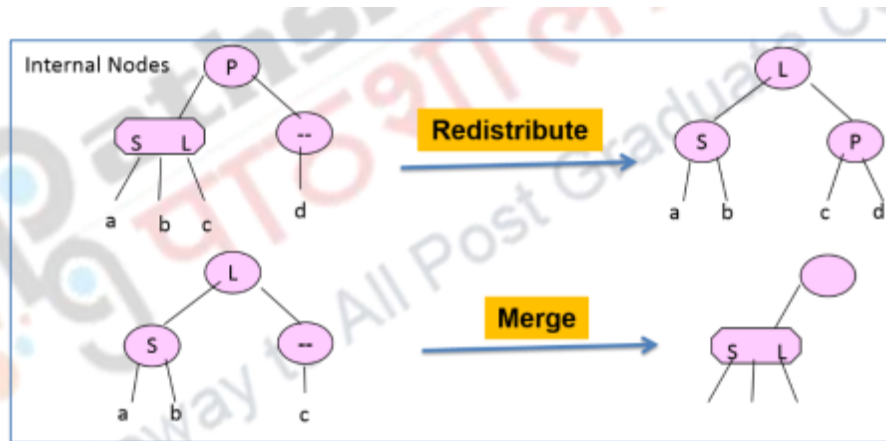


Figure 28.15 Deleting an Internal Node

28.5.3 Deleting Root

Finally let us consider the case where the root itself is deleted. In this case, the child of the parent (root) becomes the parent node and the height of the tree reduces by 1 (Figure 28.16).



Figure 28.16 Deleting the Root

28.5.4 Deletion Algorithm

Deleting key k is similar to inserting: there is a special case when T is just a single (leaf) node containing k (T is made empty); otherwise, the parent of the node to be deleted is found, then the tree is fixed up if necessary so that it is still a 2-3 tree.

Once node n (the parent of the node to be deleted) is found, there are two cases,

depending on how many children n has:

case 1: n has 3 children

- Remove the child with value k , then fix $n.\text{leftMax}$, $n.\text{middleMax}$, and n 's ancestors' leftMax and middleMax fields if necessary.

case 2: n has only 2 children

- If n is the root of the tree, then remove the node containing k . Replace the root node with the other child (so the final tree is just a single leaf node).
- If n has a left or right sibling with 3 kids, then:
 - remove the node containing k
 - “steal” one of the sibling's children
 - fix $n.\text{leftMax}$, $n.\text{middleMax}$, and the leftMax and middleMax fields of n 's sibling and ancestors as needed.
- If n 's sibling(s) have only 2 children, then:
 - remove the node containing k
 - Make n 's remaining child a child of n 's sibling
 - fix leftMax and middleMax fields of n 's sibling as needed
- remove n as a child of its parent, using essentially the same two cases (depending on how many children n 's parent has) as those just discussed

28.6 2-3 Tree vs. Binary Tree

- A 2-3 tree is not a binary tree since a node in the 2-3 tree can have three children. However a 2-3 tree does resemble a full binary tree. If a 2-3 tree does not contain 3-nodes, it is like a full binary tree since all its internal nodes have two children and all its leaves are at the same level.
- If a 2-3 tree does have three children, the tree will contain more nodes than a full binary tree of the same height. Therefore, a 2-3 tree of height h has at least $2^{h+1} - 1$ nodes if the root is at $h=0$. In other words, a 2-3 tree with N nodes never has height greater than $\log(N+1)$, the minimum height of a binary tree with N nodes.

28.7 Advantages of the 2-3 trees

Though searching a 2-3 tree is not more efficient than searching a binary search tree, by allowing the node of a 2-3 tree to have three children, a 2-3 tree might be shorter than the shortest possible binary search tree. Maintaining the balance of a 2-3 tree is relatively simple than maintaining the balance of a binary search tree. Searching a 2-3 tree is as efficient as searching the shortest binary search tree, Searching a 2-3 tree is $O(\log_2 n)$. The number of comparisons required to search a 2-3 tree for a given item is approximately equal to the number of comparisons required to search a binary search tree that is as balanced as possible (Figure 28.17).

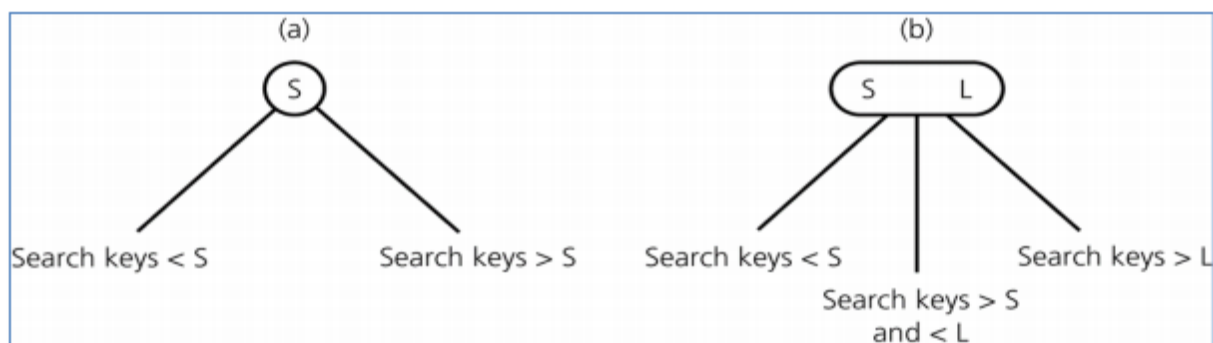


Figure 28.17 Searching a 2-3 Tree

Summary

- Explained the idea of Multi-way Trees
- Discussed the concept of 2 -3 Trees
- Described the operations on 2-3 Trees

you can view video on Multi-Way Trees and 2-3 Trees



Web Links
http://www.cs.usfca.edu/~brooks/S04classes/cs245/lectures/lecture17.pdf
http://cs.wellesley.edu/~cs230/fall04/2-3-trees.pdf
https://www.youtube.com/watch?v=bhKixY-cZHE
http://www-bcf.usc.edu/~dkempe/CS104/11-14.pdf
cobweb.cs.uga.edu/~eileen/2720/Notes/2-3Trees.ppt
http://software.ucv.ro/~mburicea/lab9ASD.pdf
https://www.cs.mtsu.edu/~jhankins/files/3110/presentations/2-3Trees.ppt
Supporting & Reference Materials
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Adam Drozdek, "Data Structures and Algorithms in C++", Cengage; 4th edition, 2013
Alfred V. Aho and Jeffrey D. Ullman, "Data Structures and Algorithms", 1983
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014